



Desenvolvimento Front-End

Prof. Elias A. da Silva
elias.asilva@sp.senac.br

HTML – Formulários

Todo sistema web tem formulário.

- Login → Campos de texto + senha;
- Cadastro → Múltiplos tipos de input;
- Busca → Campo + Botão;
- Contato → Texto, e-mail, textarea;
- Checkout → Dados pessoais + cartão;
- Filtros → checkboxes + selects

HTML5 trouxe validação nativa, sem Javascript, direto no navegador.

HTML – Formulários

Antes do HTML5, toda validação de formulário dependia de JavaScript. Hoje o próprio HTML valida e-mail, URL, número, data, campo obrigatório. Isso não substitui validação no servidor, alguém mal intencionado pode burlar o HTML, mas melhora muito a experiência do usuário comum.



HTML – Formulários – Estrutura

```
<form action="destino" method="post">
```

```
  <!-- campos aqui -->
```

```
</form>
```

action → para onde os dados são enviados (URL do servidor ou deixar vazio por enquanto);

method → **GET**: dados aparecem na URL (busca, filtros — dados não sensíveis);

POST: dados vão no corpo da requisição (login, cadastro — dados sensíveis);

HTML – Formulários – Estrutura

Por enquanto o action vai ficar vazio ou com um # porque não temos back-end ainda. Mas coloquem o atributo, é boa prática e o Validator vai cobrar. GET você já conhece sem saber: é o que aparece na barra de endereço quando você pesquisa algo no Google, veja a URL depois de pesquisar. POST é invisível, dados sensíveis nunca devem ir via GET.

Senac

HTML – Formulários – Label e input

`<label for="nome">Seu nome:</label>`

Seu nome:

`<input type="text" id="nome" name="nome">`

Regra: todo **input** precisa de um **label**. O **for** do **label** deve bater com o **id** do **input**.

Por quê?

- Clicar no **label** foca o **input** (usabilidade);
- Leitores de tela associam os dois (acessibilidade);
- W3C Validator reclama se faltar;

Senac

HTML – Formulários – Label e input

Esse vínculo entre **label** e **input** via **for/id** é o erro mais comum que vejo em formulários na internet, inclusive em sites de grandes empresas. É acessibilidade básica. Pessoa cega usando leitor de tela ouve o **label** antes de preencher o campo. Sem **label**, o leitor diz '**campo de texto**' e o usuário não sabe o que preencher.

Senac

HTML – Formulários – Prática

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aula 03 - Formulários, Tabelas e Mídias</title>
</head>
<body>
  <main>
    <section id="formulario">
      <h2>Formulário de Contato</h2>
      <form action="#" method="post">
        <label for="nome">Nome completo:</label>
        <input type="text" id="nome" name="nome">
      </form>
    </section>
  </main>
</body>
</html>
```

Senac

HTML – Formulários – Tipos de input

- `type="text"` → texto simples;
- `type="email"` → valida formato de e-mail;
- `type="password"` → oculta o que é digitado;
- `type="number"` → aceita apenas números;
- `type="tel"` → telefone (teclado numérico no mobile);
- `type="url"` → valida formato de URL;
- `type="search"` → campo de busca (aparência diferente);

HTML – Formulários – Tipos de input

O **type** não é só visual. Email com **type email** no mobile abre um teclado com **@** e **ponto**. **Tel** abre o teclado numérico. O navegador também valida: se o type for **email** e o usuário digitar sem **@**, o formulário não envia. Isso é HTML5 trabalhando por vocês sem JavaScript.

Senac

HTML – Formulários – Label e input

Complementem o código com:

```
<label for="nome">Nome completo:</label>
<input type="text" id="nome" name="nome">
<br>
<label for="email">E-mail:</label>
<input type="email" id="email" name="email">
<br>
<label for="telefone">Telefone:</label>
<input type="tel" id="telefone" name="telefone">
<br>
<label for="site">Website (opcional):</label>
<input type="url" id="site" name="site">
<br>
<!-- botão provisório – vamos melhorar no Slide 10 -->
<button type="submit">Enviar</button>
```

Senac

HTML – Formulários – Tipos de input

Testem enviar sem preencher corretamente o email, o html5 vai validar o campo do email.



HTML – Formulários – Tipos de input

<code>type="date"</code>	→ seletor de data;
<code>type="time"</code>	→ seletor de hora;
<code>type="datetime-local"</code>	→ data + hora;
<code>type="month"</code>	→ mês/ano;
<code>type="week"</code>	→ semana do ano;
<code>type="range"</code>	→ controle deslizante (slider);
<code>type="color"</code>	→ seletor de cor;
<code>type="file"</code>	→ upload de arquivo;
<code>type="hidden"</code>	→ campo invisível (dados internos);

HTML – Formulários – Tipos de input

Date e time são muito usados em sistemas de agendamento. Range aparece em filtros de preço, aquele slider de 'preço mínimo / máximo'. Color é raramente usado em formulários comuns mas aparece em ferramentas de customização. File é essencial para upload de imagens e documentos. Hidden é invisível para o usuário mas envia dados junto com o formulário , muito usado para IDs de sessão e tokens.

HTML – Formulários – Tipos de input

Insiram essa nova seção abaixo da seção de Formulário:

```
<section id="inputs-especiais">
  <h2>Tipos de Input Especiais</h2>
  <form action="#" method="get">

    <label for="nascimento">Data de nascimento:</label>
    <input type="date" id="nascimento" name="nascimento">

    <label for="horario">Horário preferido:</label>
    <input type="time" id="horario" name="horario">

    <label for="satisfacao">Nível de satisfação:</label>
    <input type="range" id="satisfacao" name="satisfacao"
      min="0" max="10" step="1" value="5">

    <label for="cor">Cor favorita:</label>
    <input type="color" id="cor" name="cor" value="#2E75B6">

    <label for="arquivo">Enviar arquivo:</label>
    <input type="file" id="arquivo" name="arquivo"
      accept=".jpg, .png, .pdf">

  </form>
</section>
```

HTML – Formulários – Tipos de input

O **accept** no **input file** filtra os tipos de arquivo no seletor do sistema operacional. **Min**, **max** e **step** do **range** controlam os limites e o incremento do **slider**.



HTML – Form Input – Atributos

- `required` → campo obrigatório;
- `placeholder="..."` → texto de dica dentro do campo;
- `value="..."` → valor pré-preenchido;
- `disabled` → campo desabilitado;
- `readonly` → leitura apenas, não editável;
- `maxlength="100"` → limite de caracteres;
- `minlength="3"` → mínimo de caracteres;
- `min="0" max="120"` → limites para number e range;
- `pattern="regex"` → valida com expressão regular;
- `autocomplete="on"` → permite sugestões do navegador;

Senac

HTML – Form Input – Atributos

Required é o mais importante. **Placeholder** não substitui **label**, é um erro muito comum colocar só **placeholder** e remover o **label**. O **placeholder** desaparece quando o usuário começa a digitar e a pessoa não sabe mais o que precisa preencher. **Label** é obrigatório. **Placeholder** é complemento. **Pattern** aceita expressão regular, por exemplo, **pattern**='[0-9]{5}-[0-9]{3}' valida formato de CEP.

HTML – Form Input – Atributos

Vamos atualizar os campos do form de contato:

```
<form action="#" method="post">

  <label for="nome">Nome completo:</label>
  <input type="text" id="nome" name="nome"
    placeholder="Ex: Maria da Silva"
    required minlength="3" maxlength="80"
    autocomplete="name">

  <br>
  <label for="email">E-mail:</label>
  <input type="email" id="email" name="email"
    placeholder="seu@email.com"
    required autocomplete="email">

  <br>
  <label for="telefone">Telefone:</label>
  <input type="tel" id="telefone" name="telefone"
    placeholder="(19) 99999-9999"
    pattern="\{0-9\}{2}\{0-9\}{4,5}-\{0-9\}{4}">

  <br>
  <label for="site">Website (opcional):</label>
  <input type="url" id="site" name="site">

  <br>
  <!-- botão provisório – vamos melhorar no Slide 10 -->
  <button type="submit">Enviar</button>

</form>
```

HTML – Form Input – TextArea, Select, Datalist

- `<textarea>` → campo de texto longo (multi-linha);
rows e cols controlam tamanho inicial;
redimensionável pelo usuário;
- `<select>` → lista suspensa de opções;
- `<option>` → cada opção da lista;
- `<optgroup>` → agrupamento de opções;
- `<datalist>` → sugestões para input text;
usuário pode digitar ou escolher;
diferente de select: não obriga escolha;

HTML – Form Input – TextArea, Select, Datalist

Datalist é pouco usado mas muito inteligente. Imagina um campo 'cidade', você pode digitar qualquer coisa OU escolher de uma lista de sugestões. **Select** obriga a escolher uma opção da lista. **Datalist** é flexível. **Optgroup** organiza **selects** com muitas opções, como um **select** de estado dentro de um país.

HTML – Form Input – TextArea, Select, Datalist

Ainda dentro da seção formulário, digitem:

```
<fieldset>
  <legend>Como prefere ser contatado?</legend>

  <input type="radio" id="contato-email"
    name="preferencia" value="email">
  <label for="contato-email">Por e-mail</label>

  <input type="radio" id="contato-telefone"
    name="preferencia" value="telefone">
  <label for="contato-telefone">Por telefone</label>

  <input type="radio" id="contato-whatsapp"
    name="preferencia" value="whatsapp" checked>
  <label for="contato-whatsapp">Por WhatsApp</label>
</fieldset>
```

```
<fieldset>
  <legend>Áreas de interesse:</legend>

  <input type="checkbox" id="int-html"
    name="interesse" value="html">
  <label for="int-html">HTML/CSS</label>

  <input type="checkbox" id="int-js"
    name="interesse" value="js">
  <label for="int-js">JavaScript</label>

  <input type="checkbox" id="int-ux"
    name="interesse" value="ux">
  <label for="int-ux">UX Design</label>
</fieldset>
```

HTML – Form Input – Botões

`<button type="submit">` → envia o formulário;

`<button type="reset">` → limpa todos os campos;

`<button type="button">` → não faz nada sozinho
ação via JavaScript;

`<input type="submit">` → alternativa ao button
menos flexível.

Prefira `<button>` — aceita HTML interno, ícones, elementos filhos.

HTML – Form Input – Botões

Button type button parece inútil mas é o mais usado em *JavaScript*, você associa uma ação a ele via ***addEventListener***. **Submit** dispara o evento **submit** do **form**. **Reset** limpa tudo, use com cuidado, usuários clicam por acidente e perdem tudo que preencheram. Por isso **reset** é raro em formulários reais.

Senac

HTML – Form Input – Botões

Agora vamos finalizar o form de contato:

```
        </fieldset>

        <button type="submit">Enviar mensagem</button>
        <button type="reset">Limpar formulário</button>

    </form>
</section>
```

Testem o reset depois de preencher todos os campos. E tentem enviar com campos obrigatórios em branco, vejam as mensagens nativas do navegador.

HTML – Tabelas

USE tabela para: → Dados tabulares reais
(horários, preços, comparativos,
planilhas, resultados)

NÃO USE tabela para: → Layout de página
(era feito nos anos 2000, hoje é CSS)

Regra prática: Se o conteúdo faz sentido num Excel, faz sentido numa tabela HTML.

HTML – Tabelas

Nos anos 2000 o layout inteiro era feito com tabelas porque CSS era fraco e inconsistente entre navegadores. Hoje isso é crime semântico, e o W3C vai reclamar. Mas tabela para dados tabulares é não só válido como recomendado. Se você tem uma lista de produtos com preço, quantidade e categoria, isso é tabela.

Senac

HTML – Anatomia de Tabelas

- `<table>` → container da tabela
- `<caption>` → título/legenda da tabela
- `<thead>` → cabeçalho (linha de títulos)
- `<tbody>` → corpo (dados)
- `<tfoot>` → rodapé (totais, resumos)

Dentro de tr (table row = linha):

- `<th>` → célula de cabeçalho (negrito + centralizado)
- `<td>` → célula de dado

`scope="col"` → th para coluna

`scope="row"` → th para linha (acessibilidade)



HTML – Anatomia de Tabelas

Caption é a legenda da tabela, diferente de **figcaption**, é específico para **table**. **Thead**, **tbody** e **tfoot** são semânticos, acessibilidade e também funcionalidade: em impressão, se a tabela ocupar várias páginas, **thead** e **tfoot** se repetem automaticamente em cada página. **Th** com **scope** é acessibilidade, o leitor de tela associa cada dado ao seu cabeçalho correto.

HTML – Tabelas

Vamos criar juntos uma nova tabela, criem uma nova section:
Como o código é enorme, sigam o snippets encaminhado com a aula.

Tabela de Tecnologias

Tecnologias do Curso de Desenvolvimento

Tecnologia	Tipo	Nível	Aulas
HTML5	Marcação	Fundamental	3
CSS3	Estilização	Fundamental	5
JavaScript	Linguagem	Intermediário	4
Bootstrap 5 Framework		Intermediário	2
Total de aulas de conteúdo			14

Senac

HTML – Colspan e Rowspan

`colspan="N"` → célula ocupa N colunas;

`rowspan="N"` → célula ocupa N linhas;

Use com planejamento, confundir as contagens quebra a tabela inteira.

Dica: desenhe no papel antes de codar células que fazem merge.

Senac

HTML – Colspan e Rowspan

Colspan e rowspan são o equivalente ao 'mesclar células' do Excel. A diferença é que você precisa contar certinho, se você diz colspan 2, precisa remover uma célula dessa linha para compensar. O erro mais comum é esquecer de remover a célula correspondente e a tabela fica torta.

Senac

HTML – Colspan e Rowspan

Acessem o snippet de Colspan|Rowspan, e repliquem o código e vejam o resultado de como ficaria uma tabela de horários, ele ocupara uma nova seção no seu código.

Horário de Aulas

Grade semanal — UC15			
Horário	Segunda	Quarta	Sexta
08:00 – 10:00	Desenvolvimento Front-End		Livre
10:00 – 12:00	Banco de Dados	Projeto Integrador	Front-End (cont.)
13:00 – 15:00	Livre		Livre

Senac

HTML – Colspan e Rowspan

Reparem que na linha onde tem **rowspan='2'**, a próxima linha só tem 2 **tds** em vez de 3, a terceira célula foi 'consumida' pelo **rowspan**. Contem as células e confirmam.



HTML – Listas

- <dl> → definition list (lista de definição);
- <dt> → definition term (termo a ser definido);
- <dd> → definition description (a definição);

- Usos reais:
- Glossário técnico;
 - FAQ (pergunta + resposta);
 - Ficha técnica de produto;
 - Metadados (autor: X, data: Y, categoria: Z);
 - Pares chave-valor visíveis;

HTML – Listas

DL é a lista mais subestimada do **HTML**. Ela aparece em todo lugar, fichas de produto, glossários, especificações técnicas, mas quase ninguém usa a tag correta, todos usam **divs** ou **ps**. Usando **dl** você está comunicando ao navegador e ao Google: isso é um par termo-definição. Semanticamente perfeito para metadados visíveis.

Senac

HTML – Listas

Criem uma nova seção no código e apliquem o código do snippet de listas, o resultado será:

Glossário Front-End

HTML

HyperText Markup Language — linguagem de marcação que estrutura o conteúdo de páginas web.

CSS

Cascading Style Sheets — linguagem que define a aparência visual dos elementos HTML.

JavaScript

Linguagem de programação que adiciona comportamento interativo às páginas web.

Responsividade

Capacidade de um layout se adaptar a diferentes tamanhos de tela sem perder usabilidade.

Semântica

Uso de tags HTML pelo seu significado real, não apenas pelo efeito visual que produzem.

HTML – Listas Aninhadas

Qualquer lista pode conter outra lista.

```
<ul>  
  <li>Item pai  
    <ul>  
      <li>Filho 1</li>  
      <li>Filho 2</li>  
    </ul>  
  </li>  
</ul>
```

HTML – Listas Aninhadas

Regra: a lista filha vai DENTRO do li pai, não depois do li pai.

Usos: menus de navegação, índices, categorias com subcategorias.



HTML – Listas Aninhadas

Menus de navegação complexos são listas aninhadas. O CSS depois transforma isso em **dropdown** horizontal ou vertical, mas a estrutura semântica é **lista** dentro de **lista**. O erro clássico é colocar a **lista filha** depois do **li** fechado, ela precisa estar dentro do **li**, antes do fechamento.

Senac

HTML – Listas Aninhadas

Agora para a prática, antes da primeira section dentro do body, acrescentem:

```
<header>
  <h1>Aula 03 – HTML Completo</h1>
  <nav>
    <ul>
      <li><a href="#formulario">Formulário</a></li>
      <li>
        <a href="#tabelas">Tabelas</a>
        <ul>
          <li><a href="#tabela">Tecnologias</a></li>
          <li><a href="#horarios">Horários</a></li>
        </ul>
      </li>
      <li><a href="#glossario">Glossário</a></li>
      <li><a href="#midias">Mídias</a></li>
    </ul>
  </nav>
</header>
```

Senac

HTML – Listas Aninhadas

Sem CSS ainda parece feio, uma lista dentro de outra com indentação. Mas a estrutura está correta. Quando chegarmos em CSS e depois em Bootstrap, esse nav vai virar um menu profissional com poucas linhas.



HTML – Mídias

```
<video controls width="640" height="360">
```

```
  <source src="video.mp4" type="video/mp4">
```

```
  <source src="video.webm" type="video/webm">
```

Seu navegador não suporta vídeo HTML5.

```
</video>
```



HTML – Mídias

Atributos úteis:

- controls → mostra os controles (play, pause, volume);
- autoplay → inicia automaticamente (evite — péssima UX);
- muted → silenciado (necessário para autoplay);
- loop → repete automaticamente;
- poster="x" → imagem mostrada antes de iniciar;
- preload → none / metadata / auto;



HTML – Mídias

Multiple source é fallback, o navegador tenta o primeiro formato, se não suportar tenta o segundo. MP4 funciona em todos os navegadores modernos, então na prática um source já basta. O texto dentro do video aparece só se o navegador for muito antigo e não suportar a tag, hoje isso é raridade. Autoplay com áudio é bloqueado por padrão nos navegadores modernos, por isso se precisar de autoplay use muted junto.

HTML – Mídias

Criem uma nova seção abaixo de tudo antes do fechamento do body:

```
<section id="midias">
  <h2>Mídias</h2>

  <h3>Vídeo</h3>
  <figure>
    <video controls width="560"
      poster="imagens/poster-video.avif"
      preload="metadata">
      <source src="videos/demo.mp4" type="video/mp4">
      Seu navegador não suporta reprodução de vídeo.
    </video>
    <figcaption>
      Demonstração de HTML5 em ação.
    </figcaption>
  </figure>
</section>
```

HTML – Mídias

O arquivo de vídeo pode não existir ainda, tudo bem, o poster vai aparecer. O importante é ver a estrutura. Quem tiver um MP4 em mãos, coloque na pasta 'videos' e teste.

Lembrando que o poster preciso estar alinhado com um arquivo que já exista na pasta imagens.



HTML – Mídias

Iremos parar por aqui, mas teremos ainda áudio, iframe, Picture e imagem responsiva.

